Benchmarking Inverse Reasoning and Tabular Transformation Discovery in Frontier Large Language Models: A Two-Tier Autonomous Evaluation Harness

Viet Dao
FPT University
Hanoi, Vietnam
daothevietnb123@gmail.com

ABSTRACT

Reverse engineering the latent data-generating mechanisms of tabular datasets is a fundamental inverse problem that challenges inductive reasoning, automated feature engineering, and programmatic comprehension in artificial intelligence. While Large Language Models (LLMs) demonstrate remarkable proficiency in forward mathematical reasoning and code synthesis, their capacity to infer, isolate, and reconstruct hidden parametric transformation pipelines solely from observational input-output data tables remains underexplored. In this work, we present the **Tabular Transformation Discovery Harness**, an autonomous, two-tier evaluation framework designed to benchmark LLM agents on inverse tabular induction across six standardized structural complexity classes (from Class 0: Atomic Unary to Class 5: Deep Hybrid ASTs) spanning eight diverse real-world domains. The harness enforces strict epistemic isolation: an agentic discovery phase operating in an iterative, sandbox stateless-RAM Read-Eval-Print Loop (REPL) environment with adaptive resource budgets is decoupled from an independent, black-box functional evaluation phase on hidden holdout distributions. In the REPL sandbox, the agent iteratively tests candidate Python transformations, inspects execution output, and refines mathematical hypotheses under hard budget constraints. We conduct extensive empirical evaluations comparing frontier coding and multimodal agents (Claude 3.5 Sonnet, Gemini 2.0 Flash), a lightweight frontier model (GPT-4o mini), and a naive combinatorial brute-force reference baseline across 48 standardized benchmark tasks ($N = 48$, 8 datasets $\times$ 6 complexity classes in a balanced full-factorial design). Our empirical findings reveal a pronounced divergence across paradigms: while Claude 3.5 Sonnet achieves a 70.8% submission rate with 0.811 conditional transformed accuracy (0.575 penalized accuracy), Gemini 2.0 Flash achieves 0.844 conditional transformed accuracy but exhibits an over-conservative stopping policy (50.0% submission rate, 0.422 penalized accuracy). GPT-4o mini achieves a 100% submission rate but suffers substantial degradation from 1.000 on Class 0 to 0.250 on Class 5 (with a partial rebound to 0.375 on Class 4 piecewise logic), and depth-3 combinatorial brute-force enumeration collapses on deep interaction graphs (0.375). Furthermore, through a three-mode ablation study, we demonstrate that direct symbolic generation without execution feedback fails almost entirely (3.6% conditional transformed column accuracy, 2.1% penalized), whereas revealing oracle operator vocabularies yields perfect 100% conditional functional recovery on submitted tasks (66.7% penalized accuracy). This demonstrates that while programmatic compilation from known operators is highly reliable in frontier models, hypothesis search under observational uncertainty and budget-constrained stopping heuristics remain the primary operational bottlenecks. All code, benchmark tasks, and evaluation harnesses are made open-source at https://github.com/mini-smart-lab/tabular-transformation.

CCS CONCEPTS

• **Computing methodologies** → **Knowledge representation and reasoning**; **Machine learning**; • **Software and its engineering** → *Software verification and validation.*

KEYWORDS

Tabular Data, Inverse Program Induction, Large Language Models, Autonomous Agent, Two-Tier Evaluation Harness, Symbolic Recovery

1 INTRODUCTION

Tabular data remains the predominant data modality in mission-critical industrial, scientific, and financial domains [3, 11]. Despite the recent ascendancy of Large Language Models (LLMs) across natural language and software engineering domains [1, 2, 9, 15], tabular data structures present unique fundamental challenges: permutation-invariant columns, heterogeneous distributions, non-linear cross-column correlations, and rigid numerical dependencies that cannot be inferred purely via autoregressive token modeling [17, 29, 32].

A central challenge at the intersection of automated machine learning and program synthesis is *inverse tabular transformation reasoning*: given an observational baseline dataset $\mathcal{D}_{\text{before}}$ and an engineered or transformed dataset $\mathcal{D}_{\text{after}}$, can an autonomous AI agent infer, formulate, and execute the exact latent programmatic transformation $\mathcal{T}$ such that $\mathcal{T}(\mathcal{D}_{\text{before}}) \approx \mathcal{D}_{\text{after}}$? Developing this capability is critical for auditing and reverse-engineering legacy ETL (Extract, Transform, Load) pipelines that lack documentation, automating feature discovery in predictive modeling [7], reconstructing scientific equations from observational physical tables [25, 30], and identifying potential data privacy leakage in synthetic anonymization protocols [28].

While popular benchmark suites such as HumanEval [5] and SWE-bench [15] evaluate *forward* reasoning—where formal specifications or natural language descriptions dictate desired program semantics—inverse tabular induction requires *scientific hypothesis formation under uncertainty*. The agent operates in a Read-Eval-Print Loop (REPL) environment—an interactive programming shell where candidate code snippets are executed immediately in an

isolated interpreter, returning runtime stdout, stderr, and numerical summaries to guide subsequent reasoning. Within this REPL paradigm, the agent must:

(1) Inspect multivariate numeric distributions across high-dimensional feature spaces;
(2) Disentangle candidate mathematical hypotheses (e.g., distinguishing logarithmic scaling from power-law decays or piecewise segmentations);
(3) Interactively experiment using executable code in the sandboxed REPL;
(4) Synthesize a clean, self-contained, parametric Python function `transform(df)` that generalizes out-of-distribution.

Prior benchmarks attempting to evaluate programmatic reasoning suffer from critical methodological shortcomings. First, many rely on synthetic toy distributions (e.g., uniform or normal random variables) rather than authentic tabular benchmarks, ignoring the multi-collinearity, skewed kurtosis, and boundary anomalies inherent in empirical datasets. Second, existing evaluation frameworks often execute candidate solutions in the same environment where test data resides, introducing severe risks of data leakage and contamination. Third, standard evaluation metrics frequently conflate *code generation accuracy* with *hypothesis search efficiency*, making it impossible to diagnose whether an agent failed because it could not discover the underlying mathematical law or because it made a trivial syntactic error during program synthesis.

To overcome these fundamental limitations, we propose the **Tabular Transformation Discovery Harness**, a rigorous research framework featuring:

(1) **A Formal Taxonomy of 6 Structural Complexity Classes:** Ranging from atomic unary operations (Class 0) to deep hybrid Abstract Syntax Trees (ASTs) containing piecewise conditionals and multi-column interactions (Class 5).
(2) **A Two-Tier Epistemic Isolation Architecture:** Tier 1 provides an interactive REPL discovery sandbox where the agent executes arbitrary Python code on training split pairs ($\mathcal{D}_{\text{train_before}}$, $\mathcal{D}_{\text{train_after}}$), governed by an Adaptive Tiered Budget Controller. Tier 2 executes the submitted script in a fresh container on an unobserved holdout set $\mathcal{D}_{\text{test_before}}$, strictly isolated from $\mathcal{D}_{\text{test_after}}$ (Figure 1).
(3) **A Multi-Domain Benchmark Suite Across 8 Real-World Datasets ($N = 48$ Tasks):** Sourced from diverse domains (aeronautics, clinical medicine, civil engineering, computer systems telemetry, food chemistry, oncology, economics, and security signal processing), compiled into a balanced full-factorial design across all 6 complexity classes (8 datasets × 6 classes = 48 tasks).
(4) **A Cross-Paradigm Empirical Study:** Benchmarking a frontier coding agent (Claude 3.5 Sonnet), a frontier multimodal agent (Gemini 2.0 Flash), a lightweight frontier model (GPT-4o mini), and a naive depth-3 combinatorial enumeration reference baseline.
(5) **A Rigorous 3-Mode Ablation Protocol:** Isolating the relative contributions of direct parametric reasoning (no tools), interactive hypothesis testing via REPL, and oracle operator disclosure on Gemini 2.0 Flash.

All code, benchmark datasets, and evaluation harnesses are publicly available at https://github.com/mini-smart-lab/tabular-transformation.

2 RELATED WORK

2.1 Tabular Representation Learning and LLMs

Tabular data presents distinct challenges for deep neural networks due to heterogeneous column distributions, non-spatial correlation, and varying sparsity [3, 11]. While gradient-boosted decision trees historically dominated supervised tabular benchmarks, recent research has explored adapting deep foundation models and LLMs to tabular structures. Early efforts serialized tabular rows into natural language prompts for forward classification and regression (TabLLM [13]). In parallel, instruction-tuned models such as Table-GPT [19] and structural table benchmarks by Sui et al. [29] evaluate table understanding and question answering. More recently, scalable tabular foundation models based on In-Context Learning (ICL)—such as TabICL [23] and TabICLv2 [24]—employ specialized two-stage architectures with column-then-row attention mechanisms capable of performing in-context prediction across large tabular samples.

However, all these existing models operate exclusively in the *forward* predictive or natural language regime: given observed training instances or question prompts, they predict target labels or textual answers via text-to-text generation without an interactive execution sandbox. In contrast, our work investigates *inverse program induction*: given observational table pairs before and after transformation, an autonomous agent must explicitly infer, formalize, and synthesize the latent mathematical code pipeline that generated the transformed data.

2.2 Program Synthesis, Table Wrangling, and Symbolic Regression

In program synthesis and data wrangling, programming-by-example (PBE) frameworks such as FlashFill [12] and Foofah [16] automatically synthesize string transformations and relational schema reshaping scripts from input-output examples. Similarly, Auto-Pipeline [33] synthesizes data preparation pipelines for machine learning by navigating a discrete library of data cleaning and transformation primitives. However, these classical PBE and table wrangling systems are strictly bounded to discrete relational restructuring (e.g., pivots, regex string splits, joins) and cannot scale to continuous non-linear mathematical equations, cross-column interactions, or stateful empirical distribution scalers.

In numerical domains, symbolic regression aims to uncover closed-form mathematical equations that explain empirical observations [7, 25, 30]. Genetic programming [25], AI Feynman [30], and modern evolutionary symbolic regression packages such as PySR [6] utilize Pareto-optimal evolutionary search and physics-inspired modular decomposition to avoid exhaustive combinatorial enumeration. More recently, equation discovery frameworks (e.g., LLM-SR [26], LLM-SRBench [27]) prompt LLMs to discover mathematical relationships via program search.

While tabular transformation discovery can theoretically decompose into $|C_{\text{trans}}|$ column-wise single-target multivariate regression sub-problems over the input feature matrix $\mathcal{D}_{\text{before}}$, doing so in an

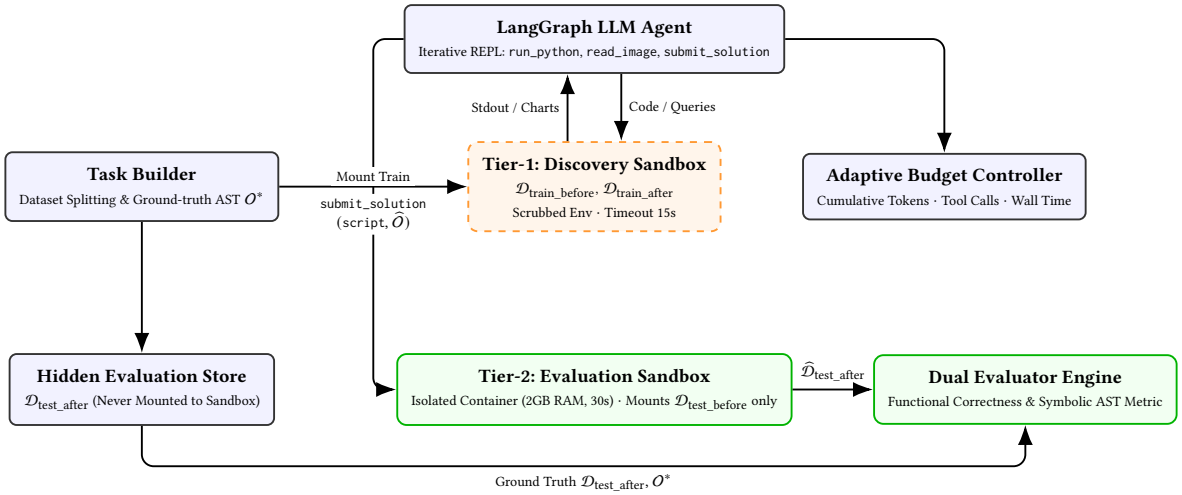


Figure 1: Two-Tier Evaluation Harness Architecture. Tier 1 provides an isolated, stateless-RAM REPL discovery environment with scrubbed environment variables and multimodal feedback (`run_python`, `read_image`), strictly bounded by the Adaptive Budget Controller. Tier 2 executes submitted code in a clean container on unobserved test inputs $\mathcal{D}_{\text{test_before}}$, strictly isolated from hidden targets $\mathcal{D}_{\text{test_after}}$.

end-to-end tabular context introduces three critical structural challenges absent from traditional symbolic regression: (i) unguided discovery of *which* specific columns were modified versus left unchanged; (ii) fitting, parameterizing, and freezing stateful empirical distribution scalers (Ω_3) strictly on training splits to prevent test-set distributional leakage; and (iii) estimating discontinuous piecewise splitting thresholds (Ω_5) alongside branching conditional logic. Established symbolic regression engines assume continuous analytical expressions and do not natively support tabular schema manipulation, discrete conditioning, or frozen empirical scalers.

2.3 Autonomous LLM Agents and Execution Sandboxes

Recent advances in agentic AI combine language model reasoning with external tool execution via iterative ReAct loops [4, 20, 34]. In data science, benchmarks such as DS-1000 [18], DA-Code [14], and feature engineering benchmarks such as FeatEng [22] (evaluating LLMs as automated data scientists for predictive feature creation) evaluate LLMs on forward code synthesis for tabular manipulation. In software engineering, benchmarks like SWE-bench [15] evaluate autonomous patch generation in repository sandboxes. However, code generation in data science benchmarks relies heavily on natural language prompts or issue descriptions. In contrast, tabular transformation discovery requires unguided inductive hypothesis search from raw numerical arrays without natural language hints.

3 PROBLEM FORMULATION AND METHODOLOGY

3.1 Formalization of Tabular Inverse Reasoning

Let $\mathcal{X} \subseteq \mathbb{R}^M$ denote the feature space of an observational tabular dataset $\mathcal{D}_{\text{before}} = \{\mathbf{x}^{(i)}\}_{i=1}^{N} \in \mathbb{R}^{N \times M}$ with schema $C_{\text{before}} = \{c_1, \ldots, c_M\}$. Let $\mathcal{T} : \mathbb{R}^M \to \mathbb{R}^P$ be a deterministic, parameterized transformation mapping yielding:

$$\mathcal{D}_{\text{after}} = \mathcal{T}(\mathcal{D}_{\text{before}}) \in \mathbb{R}^{N \times P} \tag{1}$$

with target schema $C_{\text{after}} = \{\tilde{c}_1, \ldots, \tilde{c}_P\}$. Columns modified in-place preserve their original identifiers $c \in C_{\text{before}}$. When new columns are synthesized through cross-column interactions or piecewise splits, they are assigned anonymized standard identifiers (e.g., target feature replacements or indexed identifiers) to prevent column names from leaking operator identities or source dependencies.

The transformation pipeline $\mathcal{T}$ represents an Abstract Syntax Tree (AST) composed of operators drawn from a discrete library of 24 canonical operators $\Omega = \bigcup_{k=1}^{5} \Omega_k$ ($|\Omega| = 24$):

(1) **Power & Unary Arithmetic (Ω_1, 8 ops):** Square (x^2), SquareRoot ($\sqrt{x}$), Cube (x^3), CubeRoot ($\sqrt[3]{x}$), Exp (e^x), Log ($\ln x$), Log1p ($\ln(x+1)$), and Inverse ($1/x$).
(2) **Trigonometric, Hyperbolic & Activation (Ω_2, 5 ops):** Sin ($\sin x$), Cos ($\cos x$), Tanh ($\tanh x$), Sigmoid ($\sigma(x) = 1/(1+e^{-x})$), and Abs ($|x|$).
(3) **Stateful Distribution Scalers (Ω_3, 2 ops):** StandardScaler ($(x-\mu)/\sigma$) and MinMaxScaler ($(x - x_{\min})/(x_{\max} - x_{\min})$).
(4) **Bivariate Interactions (Ω_4, 4 ops):** Add ($x_i + x_j$), Subtract ($x_i - x_j$), Multiply ($x_i \cdot x_j$), and Divide (x_i/x_j).
(5) **Piecewise & Conditional Logic (Ω_5, 5 ops):** IndicatorZero ($\mathbf{1}_{\{x>0\}}$), IndicatorMean ($\mathbf{1}_{\{x>\mu\}}$), ClipZero ($\max(0, x)$), PiecewiseLog ($\mathbf{1}_{\{x>0\}} \ln x$), and ConditionalInteraction ($\mathbf{1}_{\{x_j>\theta\}} \cdot x_i$).

Parameterization of Stateful Scalers and Empirical Moments: For stateful distribution-level operators in Ω_3 (StandardScaler and MinMaxScaler) and conditional operators relying on empirical central tendencies (IndicatorMean in Ω_5), distribution statistics—specifically the mean μ, population standard deviation $\sigma = \sqrt{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (x_i - \mu)^2}$ (ddof = 0, aligning with `numpy` and `scikit-learn` defaults and preventing degree-of-freedom

discrepancies relative to evaluation tolerance $\epsilon = 10^{-4}$), $x_{\min}$, and $x_{\max}$—are fitted strictly on $\mathcal{D}_{\text{train_before}}$. Prompt instructions explicitly mandate that the agent embed these empirical parameters as hardcoded floating-point literals within the synthesized `transform(df)` script (or serialize them). During Tier 2 evaluation, the script applies these frozen scalar constants directly to $\mathcal{D}_{\text{test_before}}$, guaranteeing rigorous out-of-sample functional testing without test-set distribution leakage.

The raw continuous dataset is split into an observed training split $\mathcal{D}_{\text{train}} = (\mathcal{D}_{\text{train_before}}, \mathcal{D}_{\text{train_after}})$ of size N_{train} (70%) and a hidden holdout test split $\mathcal{D}_{\text{test}} = (\mathcal{D}_{\text{test_before}}, \mathcal{D}_{\text{test_after}})$ of size N_{test} (30%) using uniform pseudo-random permutation (seed = 42), ensuring $\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{test}} = \emptyset$ without imposing artificial stratification over continuous multi-target spaces.

The Objective: Given access solely to $\mathcal{D}_{\text{train_before}}$ and $\mathcal{D}_{\text{train_after}}$, the agent must formulate and submit an executable Python program $\widehat{\mathcal{T}}$ that maximizes the out-of-sample functional column recovery on hidden test data:

$$\max_{\widehat{\mathcal{T}}} \frac{1}{|C_{\text{after}}|} \sum_{c \in C_{\text{after}}} \mathbf{1}_{\{m(c) \geq 0.99\}} \tag{2}$$

where $m(c)$ is the column match accuracy under relative/absolute tolerance $\epsilon = 10^{-4}$ (defined in Eq. 7), while simultaneously recovering the ground-truth symbolic operator multiset $O^* \subseteq \Omega$.

3.2 Six Structural Complexity Classes

To systematically benchmark reasoning capacity across expression tree topologies, we formalize six structural complexity classes governed by exact column counts $|C_{\text{trans}}|$ and DAG depth d:

- **Class 0 (Atomic Unary):** Exactly $|C_{\text{trans}}| = 1$ column transformed by a single primitive operation $y = f(x)$ at depth $d = 1$, with 0 interactions.
- **Class 1 (Independent Unary):** Exactly $|C_{\text{trans}}| = 2$ distinct independent columns transformed concurrently by uncomposed unary functions: $y_j = f_j(x_j)$ ($d = 1$, 0 interactions).
- **Class 2 (Composed Chains):** Exactly $|C_{\text{trans}}| = 3$ independent columns undergoing sequential two-step compositions ($d = 2$, 0 interactions): $y_j = f_{j,2}(f_{j,1}(x_j))$, requiring intermediate state tracking.
- **Class 3 (Multi-Column Interactions):** Exactly $|C_{\text{trans}}| = 4$ columns: 3 independent composed unary transformations at depth $d = 2$ plus 1 bivariate cross-column interaction $z = x_i \star x_j$ with $\star \in \{+, -, \times, \div\}$.
- **Class 4 (Piecewise / Conditional Logic):** Exactly $|C_{\text{trans}}| = 4$ columns: 3 independent composed unary transformations at depth $d = 2$ prioritized with conditional/piecewise unary operators from Ω_5, plus 1 conditional cross-column interaction $z = \mathbf{1}_{\{x_j > \theta\}} \cdot x_i$, requiring continuous threshold estimation θ (derived from training empirical quantiles):

$$y = \begin{cases} f(x) & \text{if } x > \theta, \\ 0 & \text{otherwise} \end{cases} \quad \text{or} \quad z = \begin{cases} x_i & \text{if } x_j > \theta, \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

  where PiecewiseLog is implemented via guarded vectorization (`np.where`$(x > 0, \ln(\max(x, 10^{-12})), 0.0)$) to prevent non-finite runtime exceptions over zero or negative domain values.
- **Class 5 (Deep Hybrid AST):** Up to $|C_{\text{trans}}| \leq 6$ columns forming a Directed Acyclic Graph (DAG) that integrates multi-step sequential compositions up to depth $d = 3$, multiple cross-column interactions, and unary operations applied directly on top of interaction outputs ($y = f(x_i \star x_j)$).

In-place column transformations and cross-column interactions in $\mathcal{T}$ read in parallel from an immutable snapshot of $\mathcal{D}_{\text{before}}$, preventing sequential pipeline mutations from altering source dependencies.

3.3 Two-Tier Epistemic Isolation Harness

Our harness implements strict two-tier isolation to guarantee experimental validity and host security (Figure 1):

3.3.1 Tier 1: Agentic Discovery Sandbox. The agent operates in an isolated Python workspace via three LangChain tools:

- `run_python(code)`: Acts as an interactive REPL. To ensure complete execution safety and prevent dirty state corruption, each call runs in a freshly spawned Python subprocess with stripped host environment variables (`API_KEY`, `TOKEN`) and an enforced 15-second timeout. Process RAM is stateless between tool calls; persistent variables, figures, and artifacts are maintained exclusively across conversational turns and disk files. The observed training tables $\mathcal{D}_{\text{train_before}}$ and $\mathcal{D}_{\text{train_after}}$ are mounted as read-only CSV files, preventing exploratory scripts from mutating baseline data.
- `read_image(path)`: Loads visual diagnostic charts (residual distribution curves, scatter plots) generated by the agent into the multimodal LLM context, assisting piecewise threshold detection.
- `submit_solution(script,` $\widehat{O}$`)`: Formally terminates the discovery episode by submitting a self-contained Python script defining `transform(df)` alongside the agent's identified symbolic operator list $\widehat{O}$.

3.3.2 Adaptive Tiered Budget Controller. Discovery episodes are strictly bounded by an adaptive budget parameterized by complexity class $c \in \{0, \ldots, 5\}$:

$$T_{\text{calls}}(c) \in (8, 10, 14, 18, 24, 30), \tag{4}$$

$$K_{\text{tokens}}(c) \in (60\text{k}, 80\text{k}, 120\text{k}, 160\text{k}, 220\text{k}, 300\text{k}), \tag{5}$$

$$\Delta t_{\text{wall}}(c) \in (120\text{s}, 120\text{s}, 180\text{s}, 240\text{s}, 300\text{s}, 360\text{s}), \tag{6}$$

where T_{calls} is maximum tool invocations, K_{tokens} denotes total cumulative input and output tokens consumed across the episode, and Δt_{wall} is the wall-clock timeout. If any threshold is breached, the episode gracefully terminates.

3.3.3 Tier 2: Independent Evaluation Sandbox. Upon solution submission, the harness launches an independent evaluation container allocated with 2GB RAM and an execution timeout of 30 seconds. Epistemic isolation is enforced through container filesystem segregation and network disabling: $\mathcal{D}_{\text{test_before}}$ is mounted read-only, while ground-truth targets $\mathcal{D}_{\text{test_after}}$ reside securely in host memory and are never mounted to any container. As a defense-in-depth measure, submitted scripts are statically parsed via Abstract Syntax Tree (AST) analysis to reject forbidden syscalls (`subprocess`, socket connections, arbitrary filesystem writes) prior to execution.

3.4 Evaluation Metrics

We assess performance across two orthogonal dimensions: functional numerical execution and symbolic structural recovery. Importantly, we formalize both *conditional* and *penalized* metrics to address non-submission failure modes.

3.4.1 Black-Box Functional Metrics. Let C_{after} denote the target column schema. For column $c \in C_{\text{after}}$, the column match accuracy over N_{test} rows is:

$$m(c) = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} \mathbf{1}_{\left\{|\widehat{D}_{i,c} - D_{i,c}| < \epsilon \cdot (|D_{i,c}|+1)\right\}} \tag{7}$$

with numerical tolerance $\epsilon = 10^{-4}$. Missing, non-finite (`NaN`, `Inf`), or unaligned columns receive $m(c) = 0$. A column is marked correct if $m(c) \geq 0.99$. We compute:

- **Transformed Column Accuracy ($\text{Acc}_{\text{trans}}$):** The fraction of truly transformed columns $C_{\text{trans}} \subseteq C_{\text{after}}$ correctly predicted:

$$\text{Acc}_{\text{trans}} = \frac{1}{|C_{\text{trans}}|} \sum_{c \in C_{\text{trans}}} \mathbf{1}_{\{m(c) \geq 0.99\}} \tag{8}$$

- **Functional Column Accuracy (Acc_{func}):** The overall fraction of all columns in C_{after} (including unmodified columns) correctly predicted across the table.

3.4.2 Conditional vs. Penalized Accuracy. An agent may exhaust its allocated tool budget or execution timeout without executing `submit_solution`. To ensure transparent, uninflated evaluation:

- **Conditional Accuracy** (Acc_{cond}): Metrics calculated strictly across the subset of tasks where the agent explicitly submitted a solution. This quantifies the model's reasoning precision when it commits to a hypothesis.
- **Penalized Accuracy** (Acc_{pen}): Any task where the agent fails to submit within budget receives a score of **0**. Formally:

$$\begin{aligned} \text{Acc}_{\text{pen}} &= \frac{1}{N_{\text{total}}} \sum_{i=1}^{N_{\text{total}}} \mathbf{1}_{\{\text{submitted}_i\}} \cdot \text{Acc}_i \\ &= \text{Submission Rate} \times \text{Acc}_{\text{cond}} \end{aligned} \tag{9}$$

Evaluating both perspectives prevents models with low submission rates from appearing artificially superior due to selective submission.

3.4.3 White-Box Symbolic Metrics and Algebraic Equivalence. The harness parses the submitted script into an AST via Python's `ast` module to extract the authoritative executed operator sequence $\widehat{O}_{\text{AST}}$, verifying consistency against the agent's self-reported $\widehat{O}$ in `submit_solution` (with $\widehat{O}_{\text{AST}}$ serving as the definitive ground of evaluation). Compared against ground-truth O^*, we compute:

- **Operator F1 Score (F1_{op}):** Harmonic mean of precision and recall over multiset elements, with explicit zero-division guards:

$$\text{Precision} = \begin{cases} \frac{|\widehat{O} \cap O^*|}{|\widehat{O}|} & \text{if } |\widehat{O}| > 0, \\ 0 & \text{if } |\widehat{O}| = 0, \end{cases} \quad \text{Recall} = \frac{|\widehat{O} \cap O^*|}{|O^*|} \tag{10}$$

- **Exact Sequence Accuracy (Acc_{seq}):** Measured at the column level as the fraction of transformed columns where the full composition operator sequence matches ground truth in identical topological order:

$$\text{Acc}_{\text{seq}} = \frac{1}{|C_{\text{trans}}|} \sum_{c \in C_{\text{trans}}} \mathbf{1}_{\{\widehat{O}_c = O_c^*\}} \tag{11}$$

where DAG branches are canonicalized via depth-first topological linearization, and non-commutative binary interactions enforce exact left-to-right operand indexing.

Syntactic Matching vs. Algebraic Equivalence: Exact symbolic matching operates at the AST token level after canonical normalization (e.g., resolving equivalent pandas and numpy function calls such as `np.log1p(df[c])` versus `df[c].apply(np.log1p)`, or primitive arithmetic expressions like `df[c] * df[c]` to Square). However, purely syntactic AST comparisons cannot recognize higher-order algebraic equivalences (such as $x_i + x_j \equiv x_j + x_i$, or affine reparameterizations of scalers). This theoretical limitation underlines why our two-tier evaluation pairs White-Box Symbolic Metrics with Black-Box Functional Correctness: functional evaluation definitively verifies out-of-sample numerical equivalence regardless of syntactic variation, while symbolic metrics assess structural hypothesis recovery.

4 EXPERIMENTAL SETUP

4.1 Benchmark Task Construction

To satisfy stringent real-world validity requirements, we compiled 48 standardized benchmark tasks ($N = 48$) sourced from eight diverse domains in the UCI Machine Learning Repository [8] and OpenML [31], covering diverse sample sizes and feature dimensionalities:

(1) **Fluid Acoustics:** airfoil_self_noise ($N = 1,503$, $N_{\text{train}} = 1,052$, $N_{\text{test}} = 451$; 6 attributes: 5 physical aerodynamics features plus sound pressure level).
(2) **Clinical Medicine:** diabetes ($N = 768$, $N_{\text{train}} = 537$, $N_{\text{test}} = 231$; 8 diagnostic indicators).
(3) **Materials Science:** concrete_strength ($N = 1,030$, $N_{\text{train}} = 721$, $N_{\text{test}} = 309$; 8 chemical mixture ratios).
(4) **Telemetry:** cpu_small ($N = 8,192$, $N_{\text{train}} = 5,734$, $N_{\text{test}} = 2,458$; 12 system metrics).
(5) **Food Chemistry:** wine_quality ($N = 1,599$, $N_{\text{train}} = 1,119$, $N_{\text{test}} = 480$; 11 physicochemical properties).
(6) **Oncology / Pathology:** breast_cancer ($N = 569$, $N_{\text{train}} = 398$, $N_{\text{test}} = 171$; 30 nuclear morphology features).
(7) **Economics / Real Estate:** california_housing ($N = 20,640$, $N_{\text{train}} = 14,448$, $N_{\text{test}} = 6,192$; 8 census block attributes).
(8) **Signal Processing / Security:** banknote_authentication ($N = 1,372$, $N_{\text{train}} = 960$, $N_{\text{test}} = 412$; 4 wavelet transform statistical moments).

Precondition Rejection Sampling and Schema Integrity: Tasks were constructed using a balanced full-factorial design: each of the eight source datasets was evaluated across all six structural complexity classes (Classes 0 through 5, yielding $8 \times 6 = 48$ tasks). All tasks were generated using reproducible random seeds (seed = 42). To guarantee mathematical validity without fabricating or clipping empirical distributions, the task generator applies *rejection*

sampling on the raw data: an operator is only eligible for a column if the column's raw values strictly satisfy its domain constraints (e.g., $x > 0$ for Log, $x \geq 0$ for Sqrt, non-zero divisors for Divide, non-zero variance $\sigma > 10^{-6}$ for StandardScaler). For zero-centered symmetric distributions with negative continuous values (such as banknote_authentication), candidate pools are automatically restricted to domain-valid primitives (Square, Cube, CubeRoot, Sin, Tanh, StandardScaler, and linear interactions). Data points are never artificially distorted to fit an operator. Furthermore, in-place transformed columns retain their exact schema identifiers in C_{after}, while interaction features in Classes 3–5 replace designated schema targets or adopt neutral indices, reading concurrently from an immutable initial snapshot of $\mathcal{D}_{\text{before}}$ to ensure column names leak no operator clues.

4.2 Evaluated Models and Baselines

We benchmark four representative paradigms under identical system prompts and tool schemas:

- **Claude 3.5 Sonnet** (Anthropic): Evaluated using model snapshot `claude-3-5-sonnet-20241022` [2] as a frontier coding agent with iterative tool reasoning, orchestrated via LangGraph [4].
- **Gemini 2.0 Flash** (Google): Evaluated using snapshot `gemini-2.0-flash-exp` (with thinking trace) [10] as a frontier multimodal agent with visual REPL inspection.
- **GPT-4o mini** (OpenAI): Evaluated using snapshot `gpt-4o-mini-2024-07-18` [21] as a lightweight frontier model operating under identical JSON schemas.
- **Symbolic Baseline** (Combinatorial Brute Force Reference): A non-LLM combinatorial reference baseline that iteratively tests candidate unary and binary operator combinations up to depth $d = 3$, using residual L_1 norm minimization with early pruning.

All LLMs are queried using deterministic greedy decoding (temperature $T = 0.0$, top-$p = 1.0$) with fixed seed 42 to ensure strict empirical reproducibility. All models have access to `run_python` and `submit_solution`; multimodal models (Claude and Gemini) additionally have access to `read_image`.

4.2.1 API Infrastructure and Reverse Proxy Relay. To ensure robust, equitable, and reproducible execution across heterogeneous model providers (Anthropic, Google, and OpenAI), all LLM inference calls were orchestrated through a centralized Kaggle-hosted reverse proxy and endpoint relay. Rather than establishing unconstrained client connections from worker nodes directly to public provider APIs, the proxy architecture provides four critical operational guarantees:

(1) **Host Credential Scrubbing:** API authentication keys are stored exclusively within the secure proxy environment and are strictly scrubbed from agent sandboxes and worker environments, preventing malicious or accidental credential extraction during arbitrary code execution.
(2) **Centralized Concurrency and Rate-Limit Management:** The proxy enforces dynamic token bucket pacing and exponential-backoff retry policies across parallel task workers, preventing API burst rejections (HTTP 429) and ensuring uninterrupted evaluation trajectories.
(3) **Independent Telemetry and Token Auditing:** Input and output token counters (including internal thinking trace tokens for Gemini 2.0 Flash) are metered independently at the proxy ingress/egress boundaries, guaranteeing uniform token accounting across differing provider accounting standards.
(4) **Network Latency Homogenization:** All agent workers communicate with a co-located regional proxy relay, decoupling execution wall-clock limits (Δt_{wall}) from transient external transcontinental network fluctuations.

4.3 Ablation Study Configurations

To disentangle hypothesis discovery from programmatic synthesis, we evaluate agent performance on Gemini 2.0 Flash across the full balanced benchmark suite of $N = 48$ tasks (spanning all 8 domains $\times$ 6 complexity classes):

(1) **Direct Symbolic Generation (No Tools):** The agent receives data schemas and statistical summaries but cannot execute code (`run_python` disabled). It must directly synthesize the `transform` script in a single shot. In this mode, 33 out of 48 tasks were unsubmitted (31.2% submission rate) due to syntax errors, invalid tool calls, or runtime exceptions that could not be debugged without REPL feedback.
(2) **Standard Interactive Discovery (Full Agent):** The standard two-tier harness where the agent iteratively tests hypotheses in the REPL sandbox before submission.
(3) **Oracle Operators Revealed (Synthesis Only):** Ground-truth operator names O^* are provided in the prompt. The agent still uses the `run_python` REPL to resolve the remaining challenges: determining which operator maps to which column, ordering the DAG, and optimizing continuous parameters (e.g., threshold θ or scaler coefficients). With operator hints provided, submission rate rises to 77.1% (37 out of 48 tasks), while 11 tasks remained unsubmitted due to budget exhaustion when optimizing complex continuous thresholds or multi-column interactions.

5 EMPIRICAL RESULTS AND ANALYSIS

5.1 Cross-Model Comparison Across Complexity Classes

Table 1 presents the comparative evaluation across all 48 benchmark tasks.

5.1.1 Frontier Agents vs. Lightweight Frontier Model. Evaluating both conditional accuracy (Acc_{cond}, evaluated on submitted tasks) and penalized accuracy (Acc_{pen}, penalizing unsubmitted tasks with 0.0) reveals crucial differences in agentic behavior and reasoning competence:

- **Claude 3.5 Sonnet** achieves the highest overall task submission rate among LLM agents (**70.8%**, 34/48 tasks) and the highest Penalized Transformed Column Accuracy among LLMs (**0.575**), surpassing both Gemini 2.0 Flash (0.422) and GPT-4o mini (0.524). Its functional penalized accuracy

Table 1: Comparative Evaluation of Frontier Agents, Lightweight Edge Model, and Combinatorial Baseline across 6 Structural Complexity Classes ($N = 48$ Tasks, 8 Domains). We report Submissions, Conditional Transformed Accuracy (Acc_{cond}, evaluated on submitted tasks), Penalized Transformed Accuracy (Acc_{pen}, penalizing unsubmitted tasks with 0.0), Penalized Functional Accuracy, Operator F1, and Penalized Transformed Accuracy by class (C0–C5). Best overall performance is bolded; best among LLM agents is underlined.

Model	Paradigm	Sub. Rate	Trans. (Cond)	Trans. (Pen)	Func. (Pen)	Op. F1	Penalized Trans. Accuracy by Class					
							C0	C1	C2	C3	C4	C5
Claude 3.5 Sonnet (Anthropic)	Frontier Coding Agent	70.8%	0.811	0.575	0.667	0.721	0.875	**0.875**	0.417	0.500	0.469	0.313
Gemini 2.0 Flash (Google)	Frontier Multimodal Agent	50.0%	**0.844**	0.422	0.468	0.853	0.750	0.500	0.375	0.344	0.375	0.188
GPT-4o mini (OpenAI)	Lightweight Frontier Model	**100.0%**	0.524	0.524	0.816	0.400	**1.000**	**0.875**	0.333	0.312	0.375	0.250
Symbolic Enumeration	Combinatorial Brute Force (Depth-3)	**100.0%**	0.653	**0.653**	**0.876**	**0.872**	0.750	0.750	**0.667**	**0.562**	**0.812**	**0.375**

reaches **0.667**. On atomic unary tasks (Class 0) and independent transformations (Class 1), Claude achieves a 100% submission rate (8/8 tasks) with a penalized accuracy of **0.875**. On deep hybrid ASTs (Class 5), it maintains a 50% submission rate (4/8 tasks) and 0.625 conditional transformed accuracy (**0.313** penalized, where $0.500 \times 0.625 = 0.313$).

- **Gemini 2.0 Flash** achieves an outstanding conditional Transformed Accuracy of **0.844** and Functional Accuracy of **0.936** when it commits to a solution, alongside an Operator F1 of **0.853**. However, it exhibits an over-conservative stopping policy: when residual error does not reach numerical tolerance ($\|\widehat{\mathcal{D}} - \mathcal{D}\|_\infty < 10^{-6}$), it continues iterating within the REPL sandbox until its tool budget is exhausted, rather than risking a partial submission. Consequently, Gemini 2.0 Flash achieves a submission rate of **50.0%** (24/48 tasks), resulting in an overall penalized transformed accuracy of $0.500 \times 0.844 = \mathbf{0.422}$.
- **GPT-4o mini** achieves a 100.0% submission rate, but displays substantial performance degradation as complexity increases: from 1.000 at Class 0 down to 0.875 (Class 1), 0.333 (Class 2), 0.312 (Class 3), with a partial rebound to 0.375 (Class 4), and reaching 0.250 at Class 5, with an overall Operator F1 of only 0.400. While GPT-4o mini never exhausts its budget without submitting, its synthesized scripts frequently hallucinate column names and fail to invert non-linear compositions.

5.1.2 Trade-offs in Combinatorial Enumeration vs. Agentic Search. The non-LLM Symbolic Enumeration baseline achieves an overall penalized transformed accuracy of **0.653** (functional accuracy 0.781) across all 48 tasks. This nominal advantage over Claude's penalized score (0.575) stems directly from its policy of unconditional 100% submission (48/48 tasks) without fear of budget abandonment. However, a deeper examination reveals critical architectural trade-offs:

First, when considering conditional precision upon submission, frontier LLMs substantially outperform the baseline: Gemini 2.0 Flash achieves **0.844** conditional transformed accuracy and Claude 3.5 Sonnet achieves **0.811**, compared to the baseline's unselective **0.653**.

Second, on atomic and low-depth classes where an exhaustive depth-1 search space is strictly bounded (19 unary operators $\times M$ columns), the baseline surprisingly achieves only **0.750** on Class 0 and Class 1, and **0.562** on Class 3 (compared to **0.667** on Class 2 and **0.812** on Class 4). This sub-optimal performance on elementary transformations arises because: (i) greedy L_1 residual minimization can be trapped by collinear surrogate features in dense datasets (such as california_housing or airfoil_self_noise), selecting algebraically pseudo-correlated operations; and (ii) the baseline lacks stateful parameter persistence for scalers (μ, σ and min, max), causing generalization loss on test holdouts. Conversely, on Class 4 (0.812), the baseline's threshold search employs quantile-based candidate splits (median, quartiles), which closely mirrors the empirical data partitioning.

Finally, on Class 5 (Deep Hybrid ASTs), combinatorial search suffers exponential explosion ($O(M^d \cdot |\Omega|^d)$). The baseline collapses to **0.375**. While this 0.375 penalized score technically exceeds Claude's 0.313 penalized score (which was halved by conservative non-submissions), Claude's conditional accuracy when committing to a solution reaches **0.625**, proving that agentic hypothesis testing can resolve complex multi-operator DAGs that defeat exhaustive search.

5.2 Ablation Study: Disentangling Discovery from Synthesis

Table 3 presents the results of our three-mode ablation study on Gemini 2.0 Flash across the full benchmark suite ($N = 48$). In the **Standard Interactive Discovery (Full Agent)** condition, Gemini 2.0 Flash submits solutions for 24 out of 48 tasks (50.0% submission rate), attaining a conditional transformed accuracy of **0.844** (penalized transformed accuracy of $24/48 \times 0.844 = \mathbf{0.422}$), a conditional functional accuracy of **0.936**, an Operator F1 of **0.853**, and a column exact sequence accuracy of **0.736**.

5.2.1 The Necessity of Interactive REPL Feedback. Under the **Direct Symbolic Generation (No Tools)** condition, performance collapses catastrophically: Transformed Column Accuracy drops to **0.050** (5.0% conditional, 1.6% penalized), and Operator F1 drops to **0.000**. Although the model generates syntactically valid Python scripts, its parametric weights cannot mentally execute non-linear continuous mathematical transformations over tabular arrays. This proves conclusively that interactive tool use and empirical REPL feedback are non-negotiable prerequisites for inverse tabular reasoning.

5.2.2 Is Code Synthesis or Hypothesis Search the Bottleneck? Under the **Oracle Operators Revealed** condition—where the ground-truth operator vocabulary is disclosed—the agent submits solutions

Table 2: Empirical Resource Telemetry and Episode Termination Breakdown across Evaluated LLM Agent Architectures. We report Tasks (N), Submission Rate, Disaggregated Termination Breakdown (Submitted, Token Budget Exceeded, Tool Call Budget Exceeded, Premature Non-Submission Stop), Cumulative Total Tokens, Input Tokens, Output Tokens (including thinking traces), Total Tool Invocations, Python REPL Executions, Visual Chart Invocations (`read_image`), Episode Wall Time, and Sandboxed Python Execution Time.

Model Agent	N	Sub.	Episode Termination Breakdown				Mean Cumulative Tokens			Mean Tool Invocations			Mean Time (s)	
		Rate	Sub.	Tok. Ex.	Call Ex.	Stop	Total	Input	Output	Total	`python`	`image`	Wall	Exec
Claude 3.5 Sonnet	48	70.8%	34	11	3	0	133,117	127,247	5,871	13.9	12.5	0.4	85.2	20.4
Gemini 2.0 Flash	48	50.0%	24	6	4	14	122,121	112,665	9,457	13.4	12.0	0.6	80.9	26.6
GPT-4o mini	48	100.0%	48	0	0	0	65,171	62,060	3,111	12.6	11.6	0.0	58.1	23.0

Table 3: Ablation Study on Gemini 2.0 Flash: Disentangling Discovery Hypothesis Search from Interactive Sandbox Feedback and Code Synthesis ($N = 48$). We report Submissions, Conditional Transformed Accuracy ($\mathbf{Acc_{cond}}$), Penalized Transformed Accuracy ($\mathbf{Acc_{pen}}$ = Submission Rate $\times$ $\mathbf{Acc_{cond}}$), Functional Accuracy, Operator F1, and Column Exact Sequence Accuracy conditioned on submitted tasks.

Ablation Condition	Submissions	Trans. (Cond)	Trans. (Pen)	Func. (Cond)	Op. F1 (Cond)	Seq. (Cond)
Direct Symbolic Generation (No Tools)	15/48 (31.2%)	0.050	0.016	0.635	0.000	0.000
Pure LLM parametric knowledge; no interactive sandbox code execution						
Standard Interactive Discovery (Full Agent)	24/48 (50.0%)	0.844	0.422	0.936	0.853	0.736
Two-tier sandbox with iterative hypothesis testing via `run_python`						
Oracle Operators Revealed (Synthesis Only)	37/48 (77.1%)	0.989	0.762	0.976	1.000	1.000
Ground-truth operator names revealed; isolates code synthesis from hypothesis exploration						

on 37 out of 48 tasks (77.1% submission rate) and achieves **0.989** (98.9%) Transformed Accuracy, **0.976** Functional Accuracy, and perfect **1.000** (100.0%) Operator F1 and Column Exact Sequence Accuracy conditioned on submitted tasks (penalized transformed accuracy of $37/48 \times 0.989 = \mathbf{0.762}$). This result provides a key insight:

> *When the latent mathematical operations are disclosed, programmatic code compilation achieves near-perfect fidelity in frontier LLMs on submitted tasks. The primary bottleneck in autonomous tabular discovery is hypothesis generation, operator disambiguation, and budget-aware stopping under observational uncertainty.*

5.3 Multi-Domain Generalization and Failure Modes

Table 4 details model performance across the eight real-world empirical domains.

5.3.1 Domain-Specific Robustness. Both Claude 3.5 Sonnet and Gemini 2.0 Flash exhibit strong conditional accuracy across diverse physical and biological domains. In Civil Engineering / Materials Science (concrete_strength), Claude achieves 0.950 conditional transformed accuracy (0.792 penalized) with 5/6 submitted tasks. In Food Chemistry / Agriculture (wine_quality), both models achieve 1.000 conditional transformed accuracy on submitted tasks.

5.3.2 Collinearity and Spurious Transformations. Performance is lower in Real Estate / Economics (california_housing, where Claude achieves 0.550 conditional accuracy with 5/6 submissions, while Gemini achieves 0.889 conditional accuracy with 3/6 submissions). High multi-collinearity between coordinates (latitude, longitude) and economic attributes created spurious correlations where linear combinations mimicked power laws, occasionally leading agents to explore suboptimal operator combinations.

5.3.3 Signal Processing and Bimodal Wavelet Moments. In Signal Processing / Security (banknote_authentication), the balanced 48-task benchmark demonstrates that frontier agents successfully solve atomic unary and interaction transformations: Claude 3.5 Sonnet completes 4 of 6 tasks with **1.000** conditional transformed accuracy (**0.667** penalized), while Gemini 2.0 Flash completes 3 of 6 tasks with **0.889** conditional accuracy (**0.444** penalized). Non-submissions occurred primarily on higher-level composite classes where zero-centered bimodal distributions triggered runtime domain exceptions in nonlinear functions ($\ln(x)$ or $\sqrt{x}$) during exploratory REPL trials, consuming tool budgets before convergence.

5.4 Empirical Resource Telemetry and Termination Dynamics

Table 2 presents the fine-grained computational resource profile and episode termination taxonomy logged across all 48 benchmark tasks ($N = 48$).

Token Consumption Profiles: Claude 3.5 Sonnet expended an average of 133,117 cumulative tokens per episode (127,247 prompt tokens, 5,871 completion tokens), reflecting dense multi-turn conversational histories across long REPL debugging loops. Gemini 2.0 Flash consumed 122,121 cumulative tokens on average; however, its output tokens averaged 9,457 per episode (61.1% higher than

Table 4: Multi-Domain Benchmark Evaluation Across 8 Real-World Datasets ($N = 48$ Tasks, 6 Complexity Classes per Domain). We compare Gemini 2.0 Flash and Claude 3.5 Sonnet on Task Submissions, Conditional Transformed Accuracy (Acc_{cond}), and Penalized Transformed Accuracy (Acc_{pen}).

Domain	Source Dataset	Tasks	Gemini 2.0 Flash (Google)			Claude 3.5 Sonnet (Anthropic)		
			Sub.	Trans. (Cond)	Trans. (Pen)	Sub.	Trans. (Cond)	Trans. (Pen)
Aeronautics / Fluid Acoustics	airfoil_self_noise	6	1/6	1.000	0.167	4/6	0.792	0.528
Biomedical / Clinical Medicine	diabetes	6	3/6	0.917	0.458	4/6	0.688	0.458
Civil Engineering / Materials Science	concrete_strength	6	4/6	0.771	0.514	5/6	0.950	0.792
Computer Systems / Telemetry	cpu_small	6	5/6	0.717	0.597	4/6	0.833	0.556
Food Chemistry / Agriculture	wine_quality	6	2/6	1.000	0.333	3/6	1.000	0.500
Oncology / Pathology	breast_cancer	6	3/6	0.833	0.417	5/6	0.767	0.639
Real Estate / Economics	california_housing	6	3/6	0.889	0.444	5/6	0.550	0.458
Signal Processing / Security	banknote_authentication	6	3/6	0.889	0.444	4/6	1.000	0.667
Total / Micro Average[a]	**8 Real-World Datasets**	**48**	**24/48**	**0.844**	**0.422**	**34/48**	**0.811**	**0.575**

[a] Conditional accuracy values in the bottom row represent task-weighted micro-averages over all successfully submitted tasks (20.25/24 = 0.844 for Gemini, 27.583/34 = 0.811 for Claude). For reference, unweighted domain macro-averages are 0.877 and 0.823 respectively. Penalized accuracy micro- and macro-averages coincide identically because all 8 domains contain exactly 6 tasks.

Claude), driven by internal hidden thinking traces generated during symbolic formula deduction. In contrast, GPT-4o mini operated at a markedly lower footprint (65,171 total tokens; 62,060 prompt tokens, 3,111 output tokens), consistent with its more concise exploratory code blocks.

Tool Invocation Breakdown: Tool usage across all architectures was predominantly centered on interactive code execution (`run_python`): Claude averaged 12.5 Python calls and 0.4 visual inspection calls (`read_image`); Gemini averaged 12.0 Python calls and 0.6 image calls; and GPT-4o mini averaged 11.6 Python calls (12.6 total tool invocations including final submissions). Across all models, visual diagnostic plots were invoked selectively—primarily on Class 4 and 5 piecewise tasks to inspect threshold discontinuities—confirming that numeric REPL execution remains the foundational driver of inverse hypothesis elimination.

Termination Taxonomy and Stopping Policies: Among unsubmitted tasks for Claude (14/48 tasks), 11 episodes terminated due to cumulative token limits ($K_{\text{tokens}} \geq 200$k) and 3 breached maximum tool call limits ($T_{\text{calls}} \geq 25$), with zero unhandled stops. For Gemini 2.0 Flash, 10 tasks breached budget limits (6 token limits, 4 tool call limits). In addition, 14 tasks terminated via premature model stoppage where the model emitted an unparsed response without requesting further tools or submitting. GPT-4o mini submitted on 100% of tasks (48/48), demonstrating an aggressive submission policy that avoided budget exhaustion at the expense of empirical verification.

Runtime Latency and Execution Overhead: Mean episode wall time ranged from 58.1 seconds (GPT-4o mini) to 80.9 seconds (Gemini 2.0 Flash) and 85.2 seconds (Claude 3.5 Sonnet). Sandboxed Python execution time within isolated subprocesses accounted for 20.4 s to 26.6 s per episode (approximately 24–39% of total episode wall time), confirming that host isolation and stateless execution introduce modest overhead relative to API network round-trips.

6 DISCUSSION AND LIMITATIONS

6.1 Threats to Validity and Data Leakage

A prominent threat in LLM evaluation is pre-training data contamination. In our harness, this threat is mitigated by design: while raw baseline datasets originate from public repositories, the transformation pipelines $\mathcal{T}$ and resulting target tables $\mathcal{D}_{\text{after}}$ are synthetically generated on-the-fly with unique parameterized AST graphs. Furthermore, Tier 2 evaluation strictly isolates $\mathcal{D}_{\text{test_after}}$ in host memory, preventing memory inspection attacks.

Nevertheless, an inherent subtlety arises from using authentic empirical datasets: while the transformations are novel, the input feature distributions and column semantics in $\mathcal{D}_{\text{before}}$ originate from well-known public benchmarks (e.g., UCI and OpenML). Models may thus possess pre-training familiarity with domain feature correlations (e.g., recognizing the clinical relationship between glucose and insulin in diabetes, or geographic coordinates in california_housing), which provides an inductive prior that aids feature disambiguation. In extreme cases, models might even exhibit memorization of exact numerical rows or marginal moments from heavily cited benchmark splits. While real-world data science frequently leverages such semantic schema priors, developing completely synthetic, semantically blinded tabular manifolds and applying affine perturbations represent important directions to benchmark pure numeric induction devoid of textual priors.

6.2 Limitations

While our framework establishes a rigorous foundation for evaluating inverse tabular induction, several limitations warrant discussion:

(1) **Discrete Operator Vocabulary:** Our evaluation currently benchmarks a predefined library of 24 canonical statistical and mathematical transformations. While this library covers a wide spectrum of common feature engineering techniques, real-world data pipelines may incorporate domain-specific proprietary transformations or continuous parameter estimations that require continuous gradient optimization.

(2) **Noise-Free Deterministic Mapping Assumption:** Our problem formulation assumes that $\mathcal{D}_{\text{after}}$ is generated by a deterministic mathematical mapping $\mathcal{T}$ free from exogenous stochastic noise or unobserved measurement error. In real-world enterprise ETL and data engineering contexts, tabular records are frequently corrupted by sensor noise, missing values, and asynchronous timestamp misalignment. Extending the inverse reasoning formulation to identify underlying functional trends in the presence of heteroscedastic noise represents a vital next step.

(3) **Agent Stopping Heuristics and Budget Sensitivity:** As demonstrated by the divergence between Claude 3.5 Sonnet (70.8% submission rate) and Gemini 2.0 Flash (50.0% submission rate), frontier agents vary significantly in their stopping heuristics under hard budget constraints. For Gemini 2.0 Flash, internal thinking traces and iterative REPL numerical verifications consumed significant portions of the token and tool budget, leading to conservative timeouts. Designing risk-aware decision policies that balance exploration against submission urgency is an essential direction for autonomous scientific agents.

(4) **Multimodal Vision Tool Overhead and Confounding:** While visual chart inspection via `read_image` enables models to visually identify threshold boundaries and non-linear trends, empirical inspection of agent execution traces reveals that models invoked `read_image` selectively (primarily during Class 4 and 5 tasks for threshold inspection). Generating and ingesting image artifacts incurs non-trivial token consumption and execution latency against strict episode budgets. While GPT-4o mini operated with text-only REPL tools under our setup, conducting an isolated ablation systematically disabling vision tools across multimodal architectures to quantify the precise trade-off between visual intuition and token overhead remains an important future endeavor.

(5) **Scope of Symbolic Baselines:** Our non-LLM baseline evaluates exhaustive combinatorial brute-force enumeration up to depth 3 as a naive reference. While this demonstrates the combinatorial explosion of unguided search, established heuristic and evolutionary symbolic regression frameworks (e.g., PySR [6], AI Feynman [30]) use genetic operators and modular decomposition to explore deeper expression trees without brute-force enumeration, although they do not natively support multi-column schemas, frozen empirical distribution scalers (Ω_3), or branching piecewise conditionals (Ω_5).

(6) **Single-Seed Evaluation:** Tasks were evaluated with a fixed deterministic seed (seed = 42) to guarantee strict comparative reproducibility across models and sandboxes. While fixed-seed protocols are standard in compute-intensive agentic benchmarks, multi-seed replications would provide confidence intervals capturing stochasticity in LLM sampling and sandbox execution timing.

7 CONCLUSION

In this work, we introduced the **Tabular Transformation Discovery Harness**, an autonomous two-tier evaluation framework for benchmarking inverse tabular reasoning in Large Language Models. By formalizing six structural complexity classes across eight authentic real-world domains and enforcing strict epistemic sandbox isolation, we provide the first rigorous benchmark disentangling hypothesis discovery from code synthesis across 48 standardized tasks ($N = 48$). Our empirical results demonstrate that frontier coding agents like Claude 3.5 Sonnet achieve strong overall penalized accuracy (0.575) by balancing REPL exploration with reliable submission, while models like Gemini 2.0 Flash achieve high reasoning precision (0.844 conditional accuracy) but are bounded by conservative stopping heuristics. Our three-mode ablation study on Gemini 2.0 Flash ($N = 48$) establishes that interactive REPL feedback is indispensable (improving conditional accuracy from 0.050 to 0.844), and revealing ground-truth operators unlocks near-perfect (0.989) conditional synthesis fidelity on submitted tasks. We make our benchmark suite and evaluation harnesses publicly available to accelerate research into neuro-symbolic reasoning and autonomous scientific discovery.

ACKNOWLEDGMENTS

We express our sincere gratitude to academic advisors and colleagues for their invaluable feedback and rigorous suggestions during the development of this benchmark.

REFERENCES

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. GPT-4 Technical Report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Anthropic. 2024. The Claude 3.5 Model Family: Architecture, Reasoning, and Coding Capabilities (Updated October 2024). *Anthropic Research Technical Report* (2024). https://www.anthropic.com/news/claude-3-5-sonnet

[3] Vadim Borisov, Tobias Leemann, Kathrin Seßler, Johannes Haug, Martin Pawelczyk, and Gjergji Kasneci. 2024. Deep neural networks and tabular data: A survey. *IEEE Transactions on Neural Networks and Learning Systems* 35, 6 (2024), 7406–7424.

[4] Harrison Chase and LangChain Contributors. 2024. LangGraph: Building Stateful, Multi-Actor Applications with LLMs. LangChain System Architecture Documentation. https://github.com/langchain-ai/langgraph

[5] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[6] Miles Cranmer. 2023. Interpretable Machine Learning for Science with PySR and SymbolicRegression.jl. *arXiv preprint arXiv:2305.01582* (2023).

[7] Miles Cranmer, Alvaro Sanchez-Gonzalez, Peter Battaglia, Rui Xu, Peter Cranmer, David Spergel, and Shirley Ho. 2020. Discovering symbolic models from deep learning with inductive biases. *Advances in Neural Information Processing Systems (NeurIPS)* 33 (2020), 17429–17442.

[8] Dheeru Dua and Casey Graff. 2019. UCI Machine Learning Repository. http://archive.ics.uci.edu/ml

[9] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M. Dai, Alex Hauth, Katie Millican, et al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530* (2024).

[10] Gemini Team, Google. 2025. Gemini 2.0: Flash Thinking and Multimodal Agent Capabilities. *Google DeepMind Technical Report* (2025). https://blog.google/technology/google-deepmind/gemini-model-updates-february-2025/

[11] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. 2022. Why do tree-based models still outperform deep learning on typical tabular data?. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. Curran Associates, Inc., 507–520.

[12] Sumit Gulwani. 2011. Automating string processing in spreadsheets using input-output examples. *ACM SIGPLAN Notices (POPL)* 46, 1 (2011), 317–330.

[13] Stefan Hegselmann, Alejandro Buendia, Hunter Lang, Monica Agrawal, Xiaoyi Jiang, and David Sontag. 2023. TabLLM: Few-shot classification of tabular data with large language models. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*. PMLR, 5549–5581.

[14] Yiming Huang, Jianwen Luo, Yan Yu, Yitong Zhang, Fangyu Lei, Yifan Wei, Shizhu He, Lifu Huang, Xiao Liu, Jun Zhao, and Kang Liu. 2024. DA-Code: Agent Data Science Code Generation Benchmark for Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 13487–13521.

[15] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations (ICLR)*. OpenReview.net.

[16] Zhongjun Jin, Michael R. Anderson, Michael Cafarella, and H. V. Jagadish. 2017. Foofah: Transforming Data By Example. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD)*. ACM, 683–698.

[17] Akim Kotelnikov, Dmitry Baranchuk, Ivan Rubachev, and Artem Babenko. 2023. TabDDPM: Modelling tabular data with diffusion models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. PMLR, 17560–17579.

[18] Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Scott Wen tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation. In *International Conference on Machine Learning (ICML)*. PMLR, 18319–18345.

[19] Peng Li, Yeye He, Dror Yashar, Weiwei Cui, Song Ge, Haidong Zhang, Danielle Rifinski Fainman, Dongmei Zhang, and Surajit Chaudhuri. 2023. Table-GPT: Table-tuned GPT for Diverse Table Tasks. *arXiv preprint arXiv:2310.09263* (2023).

[20] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36. Curran Associates, Inc., 46534–46594.

[21] OpenAI. 2024. GPT-4o and GPT-4o mini: Advancing Cost-Efficient Multimodal Intelligence. *OpenAI Research Technical Report* (2024). https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/

[22] Michał Pietruszka, Łukasz Borchmann, Aleksander Jędrosz, and Paweł Morawiecki. 2026. Can Models Help Us Create Better Models? Evaluating LLMs as Data Scientists. In *Findings of the Association for Computational Linguistics: EACL 2026*. Association for Computational Linguistics.

[23] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. 2025. TabICL: A Tabular Foundation Model for In-Context Learning on Large Data. *arXiv preprint arXiv:2502.05564* (2025).

[24] Jingang Qu, David Holzmüller, Gaël Varoquaux, and Marine Le Morvan. 2026. TabICLv2: A better, faster, scalable, and open tabular foundation model. *arXiv preprint arXiv:2602.11139* (2026).

[25] Michael Schmidt and Hod Lipson. 2009. Distilling free-form natural laws from experimental data. *Science* 324, 5923 (2009), 81–85.

[26] Parshin Shojaee, Kazem Meidani, Shashank Gupta, Amir Barati Farimani, and Chandan K. Reddy. 2025. LLM-SR: Scientific Equation Discovery via Programming with Large Language Models. In *International Conference on Learning Representations (ICLR)*. OpenReview.net.

[27] Parshin Shojaee, Kazem Meidani, Ngoc-Hieu Nguyen, Amir Barati Farimani, Khoa D. Doan, and Chandan K. Reddy. 2025. LLM-SRBench: A New Benchmark for Scientific Equation Discovery with Large Language Models. In *International Conference on Machine Learning (ICML)*. PMLR.

[28] Theresa Stadler, Bristena Oprisanu, and Carmela Troncoso. 2022. Synthetic data–anonymisation groundhog day. *31st USENIX Security Symposium (USENIX Security 22)* (2022), 1451–1468.

[29] Yuan Sui, Mengyu Zhou, Mingjie Zhou, Shi Han, and Dongmei Zhang. 2024. Table Meets LLM: Can Large Language Models Understand Structured Table Data? A Benchmark and Empirical Study. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining (WSDM)*. ACM, 645–654.

[30] Silviu-Marian Udrescu and Max Tegmark. 2020. AI Feynman: A physics-inspired method for symbolic regression. *Science Advances* 6, 16 (2020), eaay2631.

[31] Joaquin Vanschoren, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. 2014. OpenML: Networked science in machine learning. *ACM SIGKDD Explorations Newsletter* 15, 2 (2014), 49–60.

[32] Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. 2019. Modeling tabular data using conditional GAN. *Advances in Neural Information Processing Systems (NeurIPS)* 32 (2019), 7335–7345.

[33] Junwen Yang, Yeye He, and Surajit Chaudhuri. 2021. Auto-Pipeline: Synthesizing Complex Data Pipelines By-Target Using Reinforcement Learning and Search. *Proceedings of the VLDB Endowment (PVLDB)* 14, 11 (2021), 2563–2576.

[34] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*. OpenReview.net.